Java Badge Blueprint: Fast-Track Your Oracle Certification

Dhanush V K

Question 01:

Which is a valid print statement?

A) System.println("Welcome to Duke's Choice Shop! ")

B) System.out.println("Welcome to Duke's Choice Shop!");

C) System.out(Welcome to Duke's Choice Shop!).println;

D) System.out.println("Welcome to Duke's Choice Shop!")

E) System.out.println(Welcome to Duke's Choice Shop!)

Question 01:

Which is a valid print statement?

A) System.println("Welcome to Duke's Choice Shop! ")

B) System.out.println("Welcome to Duke's Choice Shop!");

C) System.out(Welcome to Duke's Choice Shop!).println;

D) System.out.println("Welcome to Duke's Choice Shop!")

E) System.out.println(Welcome to Duke's Choice Shop!)

Correct Answer:

B) System.out.println("Welcome to Duke's Choice Shop!");

Explanation :

System.out is a valid reference to the standard output stream.
println() is the correct method for printing a line of text.

Question 02:

You've encapsulated the Customer class's size field. The related setSize method requires an integer argument. This argument is translated into a String, which is used to set the class's size String field. You instantiate a Customer c1 in the main method, which is written in another class. How should you set the size of customer c1 from the main method?

A) c1.size = "S";

B) c1.setSize('S');

C) c1.size = 3;

D) c1.setSize(3);

Question 02:

You've encapsulated the Customer class's size field. The related setSize method requires an integer argument. This argument is translated into a String, which is used to set the class's size String field. You instantiate a Customer c1 in the main method, which is written in another class. How should you set the size of customer c1 from the main method?

A) c1.size = "S";

B) c1.setSize('S');

C) c1.size = 3;

D) c1.setSize(3);

Correct Answer:

D) c1.setSize(3);

Explanation :

The size field in the Customer class is encapsulated, meaning it is private and cannot be accessed directly.

The setSize(int size) method requires an integer argument, which it translates into a String before setting the size field.

Therefore, calling c1.setSize(3); correctly follows the method's requirement.

Question 03:

You have a Customer class with a public String field size. In your program's main method, you create a Customer instance c1. How could you consolidate this switch statement, which is used to set sizes based on an int variable measurement?

Program

```
switch(measurement){
 case 1:
 c1.size "S";
 break;
 case 2:
 c1.size = S";
 break;
 case 3:
 c1.size = S";
 break;
}
```

A)
```
switch(measurement){
 case 1: case 2: case 3:
 c1.size = "S";
 break;
}
```

B)
```
switch(measurement){
 case 1 || 2 || case 3:
 c1.size = "S";
 break;
```

C)
```
switch(measurement){
 case 1, case 2, case 3,
 c1.size = "S";
 break;
}
```

D)
```
switch(measurement){
 case 1, 2, 3:
```

Question 03:

You have a Customer class with a public String field size. In your program's main method, you create a Customer instance c1. How could you consolidate this switch statement, which is used to set sizes based on an int variable measurement?

Program

```
switch(measurement){
 case 1:
 c1.size "S";
 break;
 case 2:
 c1.size = S";
 break;
 case 3:
 c1.size = S";
 break;
}
```

A)
```
switch(measurement){
 case 1: case 2: case 3:
 c1.size = "S";
 break;
}
```

B)
```
switch(measurement){
 case 1 || 2 || case 3:
 c1.size = "S";
 break;
```

C)
```
switch(measurement){
 case 1, case 2, case 3,
 c1.size = "S";
 break;
}
```

D)
```
switch(measurement){
 case 1, 2, 3:
```

Correct Answer:

```
A) switch(measurement){
    case 1: case 2: case 3:
      c1.size = "S";
      break;
    }
```

Explanation :

In a switch statement, multiple cases can be combined by writing them consecutively (e.g., case 1: case 2: case 3:).

This means that if measurement is 1, 2, or 3, the code inside the case block will execute (c1.size = "S";).

Question 04:

You have an array of Clothing objects called items. The Clothing class has public fields for size and price. You create a loop to calculate the total price of items. How would you add an if statement to ensure an item is only added to the total if it's the same size as the customer c1? Assume the Customer class has a public String field size.

A)
```
for(Clothing c: items){
 if(c == c1)){
 //add to total
 }
}
```

B)
```
for(Clothing c: items){
 if(items[c].size.equals(c.size)){
 //add to total
 }
}
```

C)
```
for(Clothing c: items){
 if(c.size.equals(c1.size)){
 //add to total
 }
}
```

D)
```
for(Clothing c: items){
    if(c.size.equals(c1)){
        //add to total
    }
}
```

Question 04:

You have an array of Clothing objects called items. The Clothing class has public fields for size and price. You create a loop to calculate the total price of items. How would you add an if statement to ensure an item is only added to the total if it's the same size as the customer c1? Assume the Customer class has a public String field size.

A)
```
for(Clothing c: items){
 if(c == c1)){
 //add to total
 }
}
```

B)
```
for(Clothing c: items){
 if(items[c].size.equals(c.size)){
 //add to total
 }
}
```

C)
```
for(Clothing c: items){
 if(c.size.equals(c1.size)){
 //add to total
 }
}
```

D)
```
for(Clothing c: items){
    if(c.size.equals(c1)){
        //add to total
    }
}
```

Correct Answer:

```
C) for(Clothing c: items){
    if(c.size.equals(c1.size)){
        //add to total
        }
    }
```

Explanation :

c.size is a String, and c1.size is also a String.

In Java, you compare strings using .equals(), not ==, so c.size.equals(c1.size) correctly checks if the clothing item's size matches the customer's size.

This ensures only items with the same size as the customer are added to the total.

Question 05:

You write a method to calculate the average price of all Clothing items owned by a Customer. This method adds the price of all items, then divides this value by the number of items. If there are no items, your method will attempt to divide by zero. What type of exception must you guard against in this scenario?

A) NullPointerException

B) IOException

C) ArrayIndexOutOfBoundsException;

D) ArithmeticException

Question 05:

You write a method to calculate the average price of all Clothing items owned by a Customer. This method adds the price of all items, then divides this value by the number of items. If there are no items, your method will attempt to divide by zero. What type of exception must you guard against in this scenario?

A) NullPointerException

B) IOException

C) ArrayIndexOutOfBoundsException;

D) ArithmeticException

Correct Answer:

D) ArithmeticException

Explanation :

If there are no items, the method will attempt to divide the total price (0) by the number of items (0).

Question 06:

Given:

Employee[] department = new Employee[10];

What exception occurs when you try adding an eleventh employee to the department?

A) ArithmeticException

B) ArrayIndexOutOfBounds Exception;

C) NullPointerException

D) IOException

Question 06:

Given:

Employee[] department = new Employee[10];

What exception occurs when you try adding an eleventh employee to the department?

A) ArithmeticException

C) NullPointerException

B) ArrayIndexOutOfBounds Exception;

D) IOException

Correct Answer:

D) ArithmeticException

Explanation :

The statement Employee[] department = new Employee[10]; creates an array that can hold exactly 10 Employee objects, indexed from 0 to 9.

If you try to add an eleventh employee (i.e., access index 10 or higher), it exceeds the array's bounds, causing an ArrayIndexOutOfBoundsException.

Question 07:

You've overridden the toString method in the Clothing class. From which class does this method originate?

A) Customer

B) ShopApp

C) Object

D) Comparable

Question 07:

You've overridden the toString method in the Clothing class. From which class does this method originate?

A) Customer

C) Object

B) ShopApp

D) Comparable

Correct Answer:

C) Object

Explanation :

The toString() method originates from the Object class, which is the parent class of all Java classes.

When you override toString() in Clothing, you are providing a custom string representation for objects of that class.

Question 08:

According to this course, what library should you add to your project so that it can be deployed on the Oracle Cloud?

A) Mastodon

B) Helium

C) Mastodon

D) Helidon

Question 08:

According to this course, what library should you add to your project so that it can be deployed on the Oracle Cloud?

A) Mastodon

B) Helium

C) Mastodon

D) Helidon

Correct Answer:

C) Object

Explanation :

Helidon is a collection of Java libraries designed for microservices development and is optimized for cloud deployment, including Oracle Cloud.

It provides two models:
Helidon SE (lightweight, functional style)
Helidon MP (MicroProfile-based, supports Jakarta EE)

Question 09:

When a class implements to Comparable interface, what method must also be implemented?

A) compare

B) toString

C) equals

D) compareTo

Question 09:

When a class implements to Comparable interface, what method must also be implemented?

A) compare

C) equals

B) toString

D) compareTo

Correct Answer:

D) compareTo

Explanation :

When a class implements the Comparable interface in Java, it must override the compareTo method. This method is used to define the natural ordering of objects.

Question 10:

You've setup a Clothing class with a has a public double field price. You've created two instances in your program's main method where item1 is a jacket and item2 is a shirt. You've also created a double for a tax rate in the main method. How would you accurately calculate the after-tax total for two shirts and a jacket from the main method?

A) double total = (item2.price*2 + item1.price)*(1+tax);

B) double total = (item2*2 + item1* (1+tax);

C) int total = (item2.price*2 + item1.price)* (1+tax);

D) double total = item2.price+item2.price+item1.price *tax;

E) int total = item2.price+item2.price+item1.price *tax;

Question 10:

You've setup a Clothing class with a has a public double field price. You've created two instances in your program's main method where item1 is a jacket and item2 is a shirt. You've also created a double for a tax rate in the main method. How would you accurately calculate the after-tax total for two shirts and a jacket from the main method?

A) double total = (item2.price*2 + item1.price)*(1+tax);

B) double total = (item2*2 + item1* (1+tax);

C) int total = (item2.price*2 + item1.price)* (1+tax);

D) double total = item2.price+item2.price+item1.price *tax;

E) int total = item2.price+item2.price+item1.price *tax;

Correct Answer:

A) double total = (item2.price*2 +
item1.price)*(1+tax);

Explanation :

item2.price * 2 →Accounts for the price of two shirts.
item1.price →Accounts for the price of one jacket.
(1 + tax) →Applies the tax rate to the total price.

The result is stored in a double to maintain precision, since price and tax are likely to be decimal values.

Question 11:

You have a Customer class with a public String field size. In you program's main method, you create a Customer instance c1. What is printed as a result of running this code from the main method?

```
int measurement = 3;
switch(measurement){
case 3:
 c1.size = "S";
 case 6:
 c1.size = "M";
 case 9:
 c1.size = "L";
 break;
 default:
 c1.size = "X";
}
System.out.println(c1.size);
```

A) S

B) X

C) M

D) L

Question 11:

You have a Customer class with a public String field size. In you program's main method, you create a Customer instance c1. What is printed as a result of running this code from the main method?

```
int measurement = 3;
switch(measurement){
case 3:
 c1.size = "S";
 case 6:
 c1.size = "M";
 case 9:
 c1.size = "L";
 break;
 default:
 c1.size = "X";
}
System.out.println(c1.size);
```

A) S

B) X

C) M

D) L

Explanation :

Let's break down the switch statement execution:

```
int measurement = 3;
switch(measurement){
case 3:
c1.size = "S"; //  Case 3 matches, so "S" is assigned.
case 6:
c1.size = "M"; //  No `break`, so execution continues.
case 9:
c1.size = "L"; //  Again, no `break`, so execution continues.
break;       //  Breaks here.
default:
c1.size = "X"; //  This is skipped because case 3 matched.
}
System.out.println(c1.size);
```

Correct Answer:

D) L

Question 12:

How would you loop through the Clothing array items to calculate the total price of all items after tax? The Clothing class has a public double field price. Assume the double variables total and tax are already declared.

A)
```
for(Clothing c: items){
  total = c.price*(1+tax);
}
```

B)
```
for(Clothing c: items[]){
  total = total + c.price*tax;
}
```

C)
```
forEach(Clothing c in items){
  total += c.price*(1+tax);
}
```

D)
```
for(Clothing c: items){
  total = total + c.price*(1+tax);
}
```

Question 12:

How would you loop through the Clothing array items to calculate the total price of all items after tax? The Clothing class has a public double field price. Assume the double variables total and tax are already declared.

A)
```
for(Clothing c: items){
  total = c.price*(1+tax);
}
```

B)
```
for(Clothing c: items[]){
  total = total + c.price*tax;
}
```

C)
```
forEach(Clothing c in items){
  total += c.price*(1+tax);
}
```

D)
```
for(Clothing c: items){
  total = total + c.price*(1+tax);
}
```

Correct Answer:

```
for(Clothing c: items){
 total = total + c.price * (1 + tax);
}
```

Explanation :

This loop iterates through each Clothing item in the items array.

c.price * (1 + tax) calculates the after-tax price for each item.

total = total + ... accumulates the total cost correctly.

Question 13:

Given:

public Customer(String name, int measurement) {...}

How would you instantiate a Customer object using this constructor?

A) Customer c1 = new Customer("Pinky", 3);

B) Customer c1 = new Customer("Pinky", 3, items);

C) Customer c1 = new Customer("Pinky", "S", items);

D) Customer c1 = new Customer();

E) Customer c1 = new Customer("Pinky", "S");

Question 13:

Given:

public Customer(String name, int measurement) {...}

How would you instantiate a Customer object using this constructor?

A) Customer c1 = new Customer("Pinky", 3);

B) Customer c1 = new Customer("Pinky", 3, items);

C) Customer c1 = new Customer("Pinky", "S", items);

D) Customer c1 = new Customer();

E) Customer c1 = new Customer("Pinky", "S");

F) Customer c1 = new Customer("Pinky")

Correct Answer:

A) Customer c1 = new Customer("Pinky", 3);

Explanation :

The given constructor:

public Customer(String name, int measurement) {…}

Takes two parameters:

1. A String (name)
2. An int (measurement)

Question 14:

You're encapsulating the Clothing class's price field by writing a setPrice method. The price field must always be set above a certain minimum. How would you implement this method?

A)
```
public void setPrice(double price) {
        if(price < MIN_PRICE){
        this.price = MIN_PRICE;
                }
    this.price = price;
                }
```

B)
```
public void setPrice() {
 if(price < MIN_PRICE){
 price = MIN_PRICE;
 }
 else{
 price = this.price;
 }
}
```

C)
```
public void setPrice(double
        price) {
    this.price = price;
        }
```

D)
```
public void setPrice(double price) {
        if(price < MIN_PRICE){
        this.price = MIN_PRICE;
                }
            else{
        this.price = price;
                }
                }
```

Question 14:

You're encapsulating the Clothing class's price field by writing a setPrice method. The price field must always be set above a certain minimum. How would you implement this method?

A)
```
public void setPrice(double price) {
        if(price < MIN_PRICE){
        this.price = MIN_PRICE;
                }
    this.price = price;
                }
```

B)
```
public void setPrice() {
 if(price < MIN_PRICE){
 price = MIN_PRICE;
 }
 else{
 price = this.price;
 }
}
```

C)
```
public void setPrice(double
        price) {
    this.price = price;
        }
```

D)
```
public void setPrice(double price) {
    if(price < MIN_PRICE){
    this.price = MIN_PRICE;
        }
        else{
    this.price = price;
        }
        }
```

Correct Answer:

D) public void setPrice(double price) {
if(price < MIN_PRICE){
this.price = MIN_PRICE;
}
else{
this.price = price;
}
}

Explanation :

1. The method takes a double price parameter
2. It checks if price is less than MIN_PRICE:

If true, sets this.price = MIN_PRICE to ensure it meets the minimum requirement.

Otherwise, assigns this.price = price.

Question 15:

A Customer class contains an encapsulated array field of Clothing items. How would you write a getItems method in the Customer class to return these items?

A)
```
public Clothing getItems(){
 return items;
}
```

B)
```
public void getItems(){
 return items;
}
```

C)
```
public Clothing[] getItems(){
 return items;
}
```

D)
```
public void getItems(Clothing items){
 return items;
}
```

Question 15:

A Customer class contains an encapsulated array field of Clothing items. How would you write a getItems method in the Customer class to return these items?

A) public Clothing getItems(){
return items;
}

B) public void getItems(){
return items;
}

C) public Clothing[] getItems(){
return items;
}

D) public void getItems(Clothing items){
return items;
}

Correct Answer:

```
C) public Clothing[] getItems(){
 return items;
}
```

Explanation :

- The items field is an array of Clothing objects.
- To return an array, the method must have a return type of Clothing[].
- return items; correctly returns the encapsulated array.

Question 16:

You've created a Clothing class along with two instances item1 and item2. How could you create a Clothing array to store these two items?

A) Clothing items = [item1, item2];

B) Clothing[] items = {item1, item2};

C) Clothing items(item1, item2});

D) Clothing[] items(item1, item2);

Question 16:

You've created a Clothing class along with two instances item1 and item2. How could you create a Clothing array to store these two items?

A) Clothing items = [item1, item2];

B) Clothing[] items = {item1, item2};

C) Clothing items(item1, item2});

D) Clothing[] items(item1, item2);

Correct Answer:

B) Clothing[] items = {item1, item2};

Explanation :

Clothing[] items →Declares an array of Clothing objects.

{item1, item2} →Initializes the array with item1 and item2.

Question 17:

Given:

Clothing item1 = new Clothing("Blue Jacket", 20.9, "M");

How would you declare the Clothing constructor?

A) public void Clothing(String description, double price, String size) {…}

B) public Clothing(String description, double price){…}

C) public void Clothing(description, size){…}

D) public Clothing(String description, double price, String size){…}

E) public Clothing(String description, String size, double price){…}

Question 17:

Given:

Clothing item1 = new Clothing("Blue Jacket", 20.9, "M");

How would you declare the Clothing constructor?

A) public void Clothing(String description, double price, String size){…}

B) public Clothing(String description, double price){…}

C) public void Clothing(description, size){…}

D) public Clothing(String description, double price, String size){…}

E) public Clothing(String description, String size, double price){…}

Correct Answer:

D) public Clothing(String description,

double price, String size){…}

Explanation :

From the given object instantiation:

Clothing item1 = new Clothing("Blue Jacket", 20.9, "M");

"Blue Jacket" →String (description)

20.9 →double (price)

"M" →String (size)

The constructor must match this order and type:

public Clothing(String description, double price, String size) { …}

Question 18:

How would you declare the MIN_PRICE field of the Clothing class? The value of this field should never change. Accessing this field should not require you to create a Clothing instance.

A) public double MIN_PRICE = 10.0;

B) public double MIN_PRICE = 10.0;

C) public static double MIN_PRICE = 10.0;

D) public double static final MIN_PRICE = 10.0;

E)public static final double MIN_PRICE = 10.0;

Question 18:

How would you declare the MIN_PRICE field of the Clothing class? The value of this field should never change. Accessing this field should not require you to create a Clothing instance.

A) public double MIN_PRICE = 10.0;

B) public double MIN_PRICE = 10.0;

C) public static double MIN_PRICE = 10.0;

D) public double static final MIN_PRICE = 10.0;

E)public static final double MIN_PRICE = 10.0;

Correct Answer:

A) public class Customer{

String name;

}

Explanation :

public class Customer →Defines the Customer class.

String name; →Declares a String field named name.

Question 19:

How would you create a Customerclass with a String field name?

A) public class Customer{
String name;
}

B) public class Customer{
String = name;
}

C) public class Customer{
String = "name";
}

D) public class Customer{
name String;
}

Question 19:

How would you create a Customerclass with a String field name?

A) public class Customer{
String name;
}

B)public class Customer{
String = name;
}

C) public class Customer{
String = "name";
}

D) public class Customer{
name String;
}

Correct Answer:

A) public class Customer{

String name;

}

Explanation :

public class Customer →Defines the Customer class.

String name; →Declares a String field named name.

Question 20:

You have a Customer class with a public String field name. What would you write in a main method to create a Customer instance and set its name field to Pinky?

A) `Customer c1;`

B) `c1.name = "Pinky";`

C) `Customer c1 = "Pinky";`

D)
```
Customer c1 = new Customer();
c1 = "Pinky";
```

E)
```
Customer c1 = new Customer();
c1.name = "Pinky";
```

Question 20:

You have a Customer class with a public String field name. What would you write in a main method to create a Customer instance and set its name field to Pinky?

A) `Customer c1;`

B) `c1.name = "Pinky";`

C) `Customer c1 = "Pinky";`

D)
```
Customer c1 = new Customer();
c1 = "Pinky";
```

E)
```
Customer c1 = new Customer();
c1.name = "Pinky";
```

Correct Answer:

E) Customer c1 = new Customer();
c1.name = "Pinky";

Explanation :

Customer c1 = new Customer(); →Creates a new instance of the Customer class.

c1.name = "Pinky"; →Sets the public name field of c1 to "Pinky".

Thank You !